

ELI Agent Algorithms — what each of the 14 (+1) agents actually computes

Contents

Roles at a glance	1
The 14 specialist algorithms	2
How the DAG sequences the RAG (your “search → summarize → check”)	3
Aspirational next steps (genuinely novel combinations)	4

Created 2026-06-01.

Your framing is the right lens: **the DAG structures the reasoning steps; RAG is the engine that pulls the raw information during those steps.** ELI maps onto this cleanly —

- **DAG layer (structure):** the AgentOrchestrator 12-stage pipeline + the AgentBus topological layers (eli/core/dag.py) + the coding subtask DAG. This is the “search → summarize → check” skeleton.
- **RAG layer (raw information):** a *subset* of the agents are retrievers (memory, knowledge_graph, file_code) plus the orchestrator’s own HyDE→FAISS→RAG→rerank pipeline. These pull evidence.
- **Effectors / reporters:** the rest act (system, plugin) or report grounded runtime facts (capability, introspection, frontier, ...).

So there are **14 specialist agents in _ALL_AGENTS** plus the **AgentOrchestrator** that sequences them — that’s the “14/15”.

Roles at a glance

Role	Agents
Retriever (RAG engine)	memory, knowledge_graph, file_code
Effector (acts on the world)	system, plugin
Grounded reporter (read-only facts)	capability, voice, introspection, frontier, reflection, proactive, self_improvement, habit
Planner (emits structure)	orchestrator (bus-level, descriptive) + the real AgentOrchestrator

Every agent shares the same skeleton: a **relevance gate** (cheap keyword/action test → skip with confidence=0 if irrelevant, so the bus stays fast) → its algorithm → an AgentResult(ok, confidence, data). The bus runs them in **topological layers** with **per-agent hard timeouts**, then _aggregate_confidence fuses them: contribution = evidence_quality × evidence_density × learned calibration, single-agent-capped, corroboration-bonused.

The 14 specialist algorithms

1. memory — hybrid dense+sparse retriever (RAG core) · 5.0s

- **Gate:** `_eli_memory_should_run` (explicit memory actions, memory markers, or ≥ 4 -word CHAT; blocks stale context bleeding into commands).
- **Algorithm:** `recall_memory` = **FAISS vector search first** (dense), with **FTS5/LIKE keyword search** as a supplement on cold/short queries (sparse); noise-filtered (drops ELI's own reflections/assistant-insights), importance-ordered. Plus `search_conversations` (FTS5), recent turns, session summaries.
- **Out:** `{results, conv_hits, memory_context}`. The primary RAG retriever.

2. system — deterministic action effector · 8.0s

- **Gate:** `action` $\in$ `SYSTEM_ACTIONS` (≈ 90 OS/runtime actions); `LLM_ACTIONS` (`GENERATE_SCRIPT`, etc.) are **deferred** (never run inside the timed parallel phase — they'd time out + double-execute).
- **Algorithm:** delegates to `executor_enhanced.execute(action, args)` behind the security allowlist/sandbox. Confidence 0.92 ok / 0.20 fail. The hands.

3. habit — pattern-store reader · 3.0s

- **Gate:** habit/routine/schedule keywords or `OPEN_APP`.
- **Algorithm:** reads `habit_rules` (all) + `habit_events('app_launch', 14d)`; confidence by rule presence. (Frequency mining itself lives in the proactive daemon; the agent surfaces the rules + event volume.)

4. self_improvement — failure-cluster surfacer · 3.0s

- **Gate:** self/improve/fix/learn keywords or `SELF_*` actions.
- **Algorithm:** `analyze_failures(7d, clustered by occurrence_count)` + pending capability proposals. Feeds the self-upgrade loop (see `background_tasks.md`).

5. proactive — artifact reader · 3.0s

- **Gate:** proactive/morning-report/insight keywords or `PROACTIVE_*` actions.
- **Algorithm:** reads `artifacts/proactive/latest_{context,summary,action}.txt` - `PROACTIVE_STATUS`. Surfaces the background daemon's findings.

6. frontier — cross-system introspection report · 5.0s

- **Gate:** `FRONTIER_STATUS` / `ELI_IDENTITY_AUDIT` or "full system status".
- **Algorithm:** `build_frontier_status_report/eli_identity_audit` — an import + wiring matrix across runtime/memory/self/proactive/image/world/labs. Deterministic whole-system audit.

7. plugin — plugin effector · 6.0s

- **Gate:** `action` $\in$ `PLUGIN_ACTIONS` (`GET_WEATHER`, `LIST_EVENTS`, `ADD_EVENT`).
- **Algorithm:** `execute(action, args)` into the plugin layer; silent skip otherwise.

8. capability — capability-manifest reader · 6.0s

- **Gate:** capability/awareness/"what can you do" or `LIST_CAPABILITIES` etc.
- **Algorithm:** `execute(LIST_CAPABILITIES | AWARENESS_STATUS | CODE_CHANGES)` → grounded capability surface (no confabulated feature lists).

9. voice — config reporter · 5.0s

- **Gate:** voice/tts/stt/mic keywords.
- **Algorithm:** reads ELI_TTS_ENGINE/ELI_PIPER_MODEL/ELI_MUTE + probes faster_whisper import. Reports, never emits audio (that's the TTS router).

10. orchestrator (bus-level) — descriptive planner · 3.0s

- **Gate:** multi-step trigger groups (e.g. {document, save, open}) or GROUNDED_SYNTHESIS_ACTIONS.
- **Algorithm:** emits a structured plan dict (now also surfaced as the typed `ExecutionPlan`). **Excluded from confidence aggregation** (it plans, it isn't evidence). The *executing* planner is the ReAct loop in `AgentOrchestrator`.

11. file_code — source-code RAG (grep retriever) · 4.0s

- **Gate:** grounded query or code/file/pipeline/db keywords.
- **Algorithm:** builds query-derived regex patterns, **greps a canonical files_map** of ELI's own modules (`agent_bus`, `engine`, `orchestrator`, `memory`, `router`, ...), returns `file:line: content` snippets. Lets ELI answer "which file does X" from real source, not memory.

12. reflection — reflection-log reader · 4.0s

- **Gate:** reflection/pattern/noticed/insight keywords or grounded query.
- **Algorithm:** recent observations (≤ 8) + session summaries (≤ 3) → insight list. ELI's "what I've noticed about how you use me."

13. introspection — self-model reporter · 4.0s

- **Gate:** architecture/pipeline/"how do you work" or `EXPLAIN*_RUNTIME`.
- **Algorithm:** `IntrospectionAgent` → `get_pipeline()` + `get_memory_stats()` + `get_runtime()` → grounded pipeline description. Powers honest self-explanation.

14. knowledge_graph — graph retriever (structured RAG) · 3.0s

- **Gate:** non-empty KG.
- **Algorithm:** `context_for_prompt(query)` over `kg_entities/kg_relations` (FTS5 fuzzy entity match + relation context). **DAG edge memory → knowledge_graph:** it now seeds the query with `intent["upstream"]["memory"]` hits (topological layering), so the graph lookup is conditioned on what memory just surfaced.

How the DAG sequences the RAG (your "search → summarize → check")

1. **Route** → action + reasoning mode.
2. **AgentOrchestrator** (non-quick) runs the staged pipeline; for non-CHAT it dispatches the **bus** then runs a **ReAct loop** (tool → observe → decide → next tool) — the executing DAG of steps.
3. **AgentBus** builds the **agent DAG** (`_AGENT_DEPENDENCIES`), runs agents in **topological layers**, passing each layer's results downstream (`memory` → `knowledge_graph`). Retrievers pull evidence; reporters ground it; effectors act.
4. **Grounding/evidence layer** gates the synthesis (`output_violates_evidence`).
5. **Coding tasks** decompose into a **subtask DAG** (`plan_graph`) and heavy ones run on **background workers** — see `dag.md` + `background_tasks.md`.

That is the literal realisation of "DAG structures the autonomous reasoning steps while RAG pulls the raw information."

Aspirational next steps (genuinely novel combinations)

These are the “never-before-put-together” moves the current architecture is one step away from:

1. **Bias/verification agent as an explicit DAG layer.** Add a verify/critic agent that depends on the retrievers (depends_on: {memory, knowledge_graph, file_code, web}) and runs *after* them — literally “search → summarize → **check for bias/contradiction**” as a topological layer, feeding the grounding gate. The DAG engine already supports it; just declare the edge + agent.
2. **A web retriever agent** (WEB_SEARCH as a first-class RAG node) so external retrieval joins memory/KG/file_code in the same layer, with the verify layer cross-checking sources.
3. **Calibration-aware selection.** The bus already learns per-(agent,action) calibration; feed it back into *selection* so chronically-zero agents are dropped from the DAG for that action (closes the learned-trust/unlearned- selection gap noted in orchestration_and_agents.md).
4. **Bus↔coding bug-memory unification.** The coding engine’s bug_memory and the agents’ self_improvement failure log are two halves of one “what went wrong and how we fixed it” store — merging them gives ELI a single experiential memory across chat and code.
5. **Orchestrator stage-dict → typed ExecutionPlan DAG.** Fold the engine’s rich stage plan into ExecutionPlan.steps as a true DAG so one plan type drives the whole pipeline end-to-end.